



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09.2

NOMBRE COMPLETO: EMILIANO GARCIA OLVERA

N° de Cuenta: 314256009

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 23/MARZO /2026

CALIFICACIÓN: _____

Introducción

En esta práctica integramos los conceptos de animación compleja, textura animada, jerarquía y blending para construir una pequeña máquina inspirada en la Eolípila (Aeolipile) de Herón de Alejandría. La escena consta de un fuego que calienta el aeolípila, el cual genera humo en tres estados (no sale, sale y crece, decrece hasta apagarse). Cuando el humo alcanza su máximo, activa un brazo palanca que lanza una esfera metálica con física de tiro parabólico y dos rebotes; en el tercer bote la esfera entra en un hueco del piso y reaparece de nuevo en el brazo, reiniciando el ciclo.

A diferencia de prácticas anteriores donde la animación se basaba principalmente en banderas y condicionales, aquí utilizamos funciones, máquinas de estados, interpolación e integración numérica de la física para coordinar varios objetos al mismo tiempo. También fue necesario corregir y extender la clase Sphere para que generara vértices con posición, UV y normales compatibles con el shader de iluminación, y se requirió depurar problemas de renderizado con la textura del hueco y la visibilidad de la esfera proyectil.

Ejercicio 1: Humo en sus 3 estados (textura animada + blending)

El humo es el motor de toda la animación. La rúbrica pide tres estados claros: no sale al inicio, se calienta el Aeolípila y empieza a salir, crece después de un tiempo, y al apagarse el fuego va decreciendo hasta ya no salir.

Edición de la textura del humo

Se utilizó una textura PNG de humo en escala de grises con canal alfa. Los bordes son transparentes y el centro ligeramente opaco, de modo que al aplicar blending las partículas se mezclan sin mostrar bordes recortados. Esto es especialmente importante al apilar varias partículas.

Blending y carga de la textura

La textura se carga con LoadTextureA() (que preserva el canal alfa) y antes de renderizar el humo se activa el blending con la función estándar SRC_ALPHA / ONE_MINUS_SRC_ALPHA:

```
humo_texture = Texture("Textures/humo.png");
humo_texture.LoadTextureA();

glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE); // NO escribir al depth buffer

// Renderizado de humo

glDepthMask(GL_TRUE); // restaurar
glDisable(GL_BLEND);
```

Máquina de estados del humo

El comportamiento se controla con una variable entera estado_humo y una intensidad continua humo_intensidad (0 a 1). La intensidad determina cuántas partículas se renderizan y qué tan grandes son; el estado controla cómo cambia la intensidad con el tiempo:

```
if (estado_humo == 1) {
    // Esperando que se caliente el aeolipile
    humo_intensidad = 0.0f;
    if (ciclo_timer > 1.5f) { // despues de 1.5s comienza a salir humo
        estado_humo = 2;
    }
}
else if (estado_humo == 2) {
    // Sale y crece
    humo_intensidad += dt * 0.4f;
    if (humo_intensidad >= 1.0f) {
        humo_intensidad = 1.0f;
        // Cuando el humo esta al maximo, activa el brazo
        if (estado_brazo == 0) {
            estado_brazo = 1; // pasa a "cargando" -> pronto lanzara
        }
    }
}
else if (estado_humo == 3) {
    // Decece hasta apagarse
    humo_intensidad -= dt * 0.5f;
    if (humo_intensidad <= 0.0f) {
        humo_intensidad = 0.0f;
        estado_humo = 1;
        ciclo_timer = 0.0f; // reiniciar ciclo
    }
}
}
```

Animación de la textura (scroll UV)

Para que el humo no se vea estático, la textura se desplaza usando un offset que se actualiza cada frame y se pasa al shader como uniform vec2:

```
// scroll de la textura (textura animada del humo)
humo_offset_v += dt * 0.4f;
humo_offset_u += dt * 0.1f;

// Dentro del loop de particulas
glm::vec2 toffsetHumo = glm::vec2(humo_offset_u + fase * 0.3f, humo_offset_v * 0.5f);

glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffsetHumo));
```

Sistema de partículas con algoritmo

El número de partículas varía con la intensidad. Cada una se coloca a una altura, escala y desplazamiento lateral diferentes, calculados con funciones senoidales para crear un patrón en zigzag natural:

```
int num_particulas = (int)(humo_intensidad * 8) + 4;

glEnable(GL_BLEND);
```

```

glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE); // NO escribir al depth buffer

// Punto de emision: justo arriba de la esfera del aeolipile
glm::vec3 emisor = POS_AEOLIPILE + glm::vec3(0.0f, 5.7f, -0.1f);

for (int p = 0; p < num_particulas; p++) {
    float fase = (float)p / (float)num_particulas;
    float altura_p = fmod(humo_offset_v + fase, 1.0f); // 0 a 1
    float escala_p = (0.5f + altura_p * 1.2f) * humo_intensidad;
    // dispersion lateral en zigzag amplio (crece con la altura)
    float disp = 0.1f + altura_p * 0.5f;
    float lateral_x = disp * sin((humo_offset_v + fase) * 6.28f * 1.3f);
    float lateral_z = disp * cos((humo_offset_v + fase) * 6.28f * 1.7f);

    model = glm::mat4(1.0);
    model = glm::translate(model, emisor + glm::vec3(lateral_x, altura_p *
4.0f, lateral_z));
    // rotacion individual de cada particula para variedad
    model = glm::rotate(model, (fase * 360.0f + humo_offset_v * 100.0f) *
toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(escala_p, escala_p, escala_p));

    // offset de textura (textura animada del humo)
    glm::vec2 toffsetHumo = glm::vec2(humo_offset_u + fase * 0.3f,
humo_offset_v * 0.5f);
    glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffsetHumo));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    color = glm::vec3(1.0f, 1.0f, 1.0f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    humo_texture.UseTexture();
    Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
    meshList[7]->RenderMesh();
}

```



Ejercicio 2: Brazo / Palanca de lanzamiento

El brazo conecta mecánicamente el humo con la esfera: cuando el humo está al máximo, el brazo se tensa y luego lanza la esfera describiendo un arco. La rúbrica indica que el humo crecido debe iniciar el movimiento del brazo y que debe haber animación clara del lanzamiento.

Geometría del brazo

El brazo se construyó directamente en el código como un paralelepípedo (cubo alargado) con 6 caras, vértices, normales y coordenadas UV explícitas. El extremo lanzador queda en $x=2.5$ en coordenadas locales, lo que permite calcular la posición de la esfera al soltarla. Para que el brazo apunte hacia el aeolípila se aplica una escala de -1 en X:

```
model = glm::mat4(1.0);
model = glm::translate(model, POS_BRAZO_BASE);
// Reflejar en X para que el brazo apunte hacia el aeolípila (hacia X negativo)
model = glm::scale(model, glm::vec3(-1.0f, 1.0f, 1.0f));
// Rotación del lanzamiento (alrededor de Z): -45 reposo, +60 lanzar
model = glm::rotate(model, brazo_angulo * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model; // matriz padre del brazo (para la esfera cargada)
```

Máquina de estados del brazo

El brazo tiene cuatro estados: reposo (-45°), cargando (-65°, tensándose), lanzando (+60°, brusco) y regresando. La velocidad de interpolación es distinta para que el lanzamiento se sienta brusco y el regreso suave:

```
if (estado_brazo == 0) {
    // reposo, esperando humo
    brazo_angulo_objetivo = -45.0f;
}
else if (estado_brazo == 1) {
    // cargando: aun mas atras, "tensandose"
    brazo_angulo_objetivo = -65.0f;
    // Cuando ya se cargo (angulo cerca del objetivo), pasa a lanzar
    if (fabs(brazo_angulo - brazo_angulo_objetivo) < 2.0f) {
        estado_brazo = 2; // lanzar!
    }
}
else if (estado_brazo == 2) {
    // LANZAR: sube rapido a +60 grados
    brazo_angulo_objetivo = 60.0f;
    // Justo cuando el brazo esta cerca de la vertical, soltar la esfera
    if (brazo_angulo > 30.0f && !esfera_lanzada) {
        esfera_lanzada = true;
        estado_esfera = 1; // volando
    }
    // Posicion inicial de la esfera = extremo del brazo (largo 2.5, con scale -1 en X)
    float ang = brazo_angulo * toRadians;
    float largo_brazo = 2.5f;
    esfera_x = POS_BRAZO_BASE.x - largo_brazo * cos(ang);
    esfera_y = POS_BRAZO_BASE.y + largo_brazo * sin(ang) + 0.4f;
    esfera_z = POS_BRAZO_BASE.z;
```

```

// Estrategia: 2 rebotes naturales + 3er bote cae en el hueco.
// El primer impacto ocurre a ~1/3 de la distancia al hueco,
// el segundo a ~2/3, y el tercero coincide con el hueco.
float piso_y = -1.6f;
glm::vec3 primer_impacto = glm::vec3(
    esfera_x + (POS_HUECO.x - esfera_x) * 0.33f,
    piso_y,
    esfera_z + (POS_HUECO.z - esfera_z) * 0.33f
);
float dx1 = primer_impacto.x - esfera_x;
float dz1 = primer_impacto.z - esfera_z;
float dy1 = primer_impacto.y - esfera_y;
float t1 = 1.1f;
esfera_vx = dx1 / t1;
esfera_vz = dz1 / t1;
esfera_vy = (dy1 + 0.5f * GRAVEDAD * t1 * t1) / t1;
}
if (brazo_angulo > 55.0f) {
    estado_brazo = 3; // regresar
}
}
else if (estado_brazo == 3) {
    // Regresar suavemente a reposo
    brazo_angulo_objetivo = -45.0f;
    if (fabs(brazo_angulo - brazo_angulo_objetivo) < 2.0f) {
        estado_brazo = 0;
    }
}
// Suavizado del angulo del brazo (interpolacion lineal)
float velocidad_brazo = (estado_brazo == 2) ? 8.0f : 2.5f;
brazo_angulo += (brazo_angulo_objetivo - brazo_angulo) * dt * velocidad_brazo;

```

Suelta de la esfera y cálculo de velocidad inicial

La esfera se suelta cuando el brazo supera 30° durante el estado de lanzamiento. La posición inicial se calcula aplicando la matriz del brazo (modelaux) al punto extremo local (2.5, 0, 0). La velocidad inicial apunta al primer tercio del camino hacia el hueco para dejar espacio a dos rebotes naturales antes de entrar al hueco:

```

if (brazo_angulo > 30.0f && !esfera_lanzada) {
    esfera_lanzada = true;
    estado_esfera = 1; // volando

    // Posicion inicial de la esfera = extremo del brazo (largo 2.5, con
    scale -1 en X)
    float ang = brazo_angulo * toRadians;
    float largo_brazo = 2.5f;
    esfera_x = POS_BRAZO_BASE.x - largo_brazo * cos(ang);
    esfera_y = POS_BRAZO_BASE.y + largo_brazo * sin(ang) + 0.4f;
    esfera_z = POS_BRAZO_BASE.z;

    // Estrategia: 2 rebotes naturales + 3er bote cae en el hueco.
    // El primer impacto ocurre a ~1/3 de la distancia al hueco,
    // el segundo a ~2/3, y el tercero coincide con el hueco.
    float piso_y = -1.6f;
    glm::vec3 primer_impacto = glm::vec3(
        esfera_x + (POS_HUECO.x - esfera_x) * 0.33f,
        piso_y,

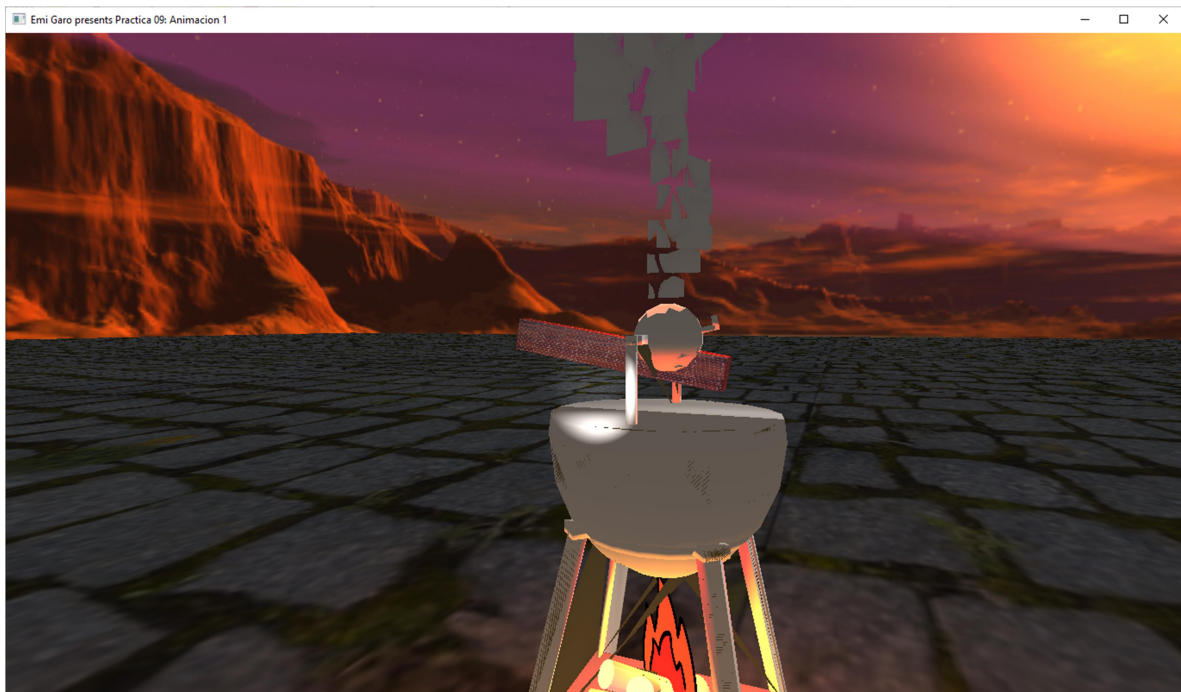
```

```

        esfera_z + (POS_HUECO.z - esfera_z) * 0.33f
    );
    float dx1 = primer_impacto.x - esfera_x;
    float dz1 = primer_impacto.z - esfera_z;
    float dy1 = primer_impacto.y - esfera_y;

    float t1 = 1.1f;
    esfera_vx = dx1 / t1;
    esfera_vz = dz1 / t1;
    esfera_vy = (dy1 + 0.5f * GRAVEDAD * t1 * t1) / t1;
}

```



Ejercicio 3: Esfera con tiro parabólico, 2 rebotes y hueco

La esfera implementa física básica (gravedad, integración de Euler, rebote inelástico) con comportamiento en dos fases: dos rebotes naturales seguidos de un tercer bote dirigido al hueco. La rúbrica pide los cuatro estados de la esfera bien implementados con buena velocidad.

Corrección de la clase Sphere

La clase Sphere original tenía tres problemas críticos que impedían que la esfera se viera en pantalla:

- `init()` y `load()` no se llamaban automáticamente: el constructor no generaba ni subía datos a la GPU, dejando el VAO vacío.

- Layout incompatible con el shader de iluminación: la Sphere usaba VertexColor (posición + color RGB), pero el shader espera (posición, UV, normal). Los atributos leían basura y la esfera se descartaba.
- Offsets incorrectos: los tres atributos apuntaban al offset 0, por lo que posición, UV y normal leían los mismos bytes.

La solución fue llamar `init()` y `load()` en el main después de crear la ventana OpenGL, y usar el layout correcto de 8 floats por vértice (x,y,z, u,v, nx,ny,nz):

```
esfera_proyectil.init();
esfera_proyectil.load();
```

Tiro parabólico (integración de Euler)

El movimiento durante el vuelo se calcula con integración numérica de Euler: en cada frame la velocidad vertical pierde $g \cdot dt$ por la gravedad, y la posición se actualiza con $v \cdot dt$:

```
if (estado_esfera == 1) {
    // Volando: aplicar fisica
    esfera_vy -= GRAVEDAD * dt;
    esfera_x += esfera_vx * dt;
    esfera_y += esfera_vy * dt;
    esfera_z += esfera_vz * dt;

    // Rebote contra el piso (y = -1.6 para que la esfera de radio 0.4 toque y=-2)
    float piso = -1.6f;
    if (esfera_y < piso && esfera_vy < 0.0f) {
        esfera_y = piso;
        esfera_vy = -esfera_vy * COEF_REBOTE;
        esfera_vx *= 0.92f;
        esfera_vz *= 0.92f;
        rebotes++;
        // En el segundo rebote, corregir rumbo directo al hueco
        // para que el tercer bote caiga exactamente en el
        if (rebotes == 2) {
            float dx = POS_HUECO.x - esfera_x;
            float dz = POS_HUECO.z - esfera_z;
            float t_corto = 0.6f;
            esfera_vx = dx / t_corto;
            esfera_vz = dz / t_corto;
            esfera_vy = (0.5f * GRAVEDAD * t_corto * t_corto) / t_corto;
        }
        // arco bajo
    }
}
```

Dos rebotes naturales + tercer bote dirigido al hueco

Los dos primeros rebotes son completamente naturales: la velocidad vertical se invierte con coeficiente de restitución 0.6 y las horizontales se frenan ligeramente por fricción. En el segundo rebote se corrige el rumbo directamente hacia el hueco para garantizar que el tercer bote caiga en él. La detección del hueco solo se activa después del segundo rebote:

```
float piso = -1.6f;
```

```

if (esfera_y < piso && esfera_vy < 0.0f) {
    esfera_y = piso;
    esfera_vy = -esfera_vy * COEF_REBOTE;
    esfera_vx *= 0.92f;
    esfera_vz *= 0.92f;
    rebotes++;

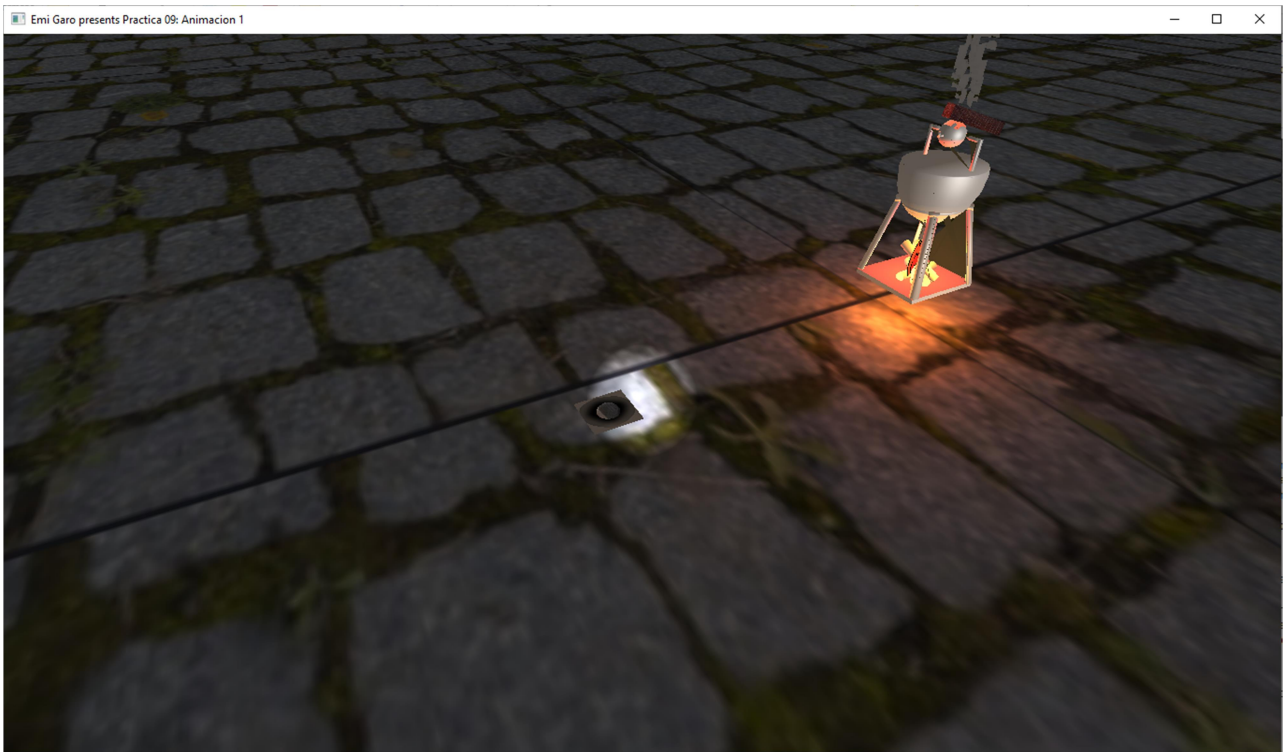
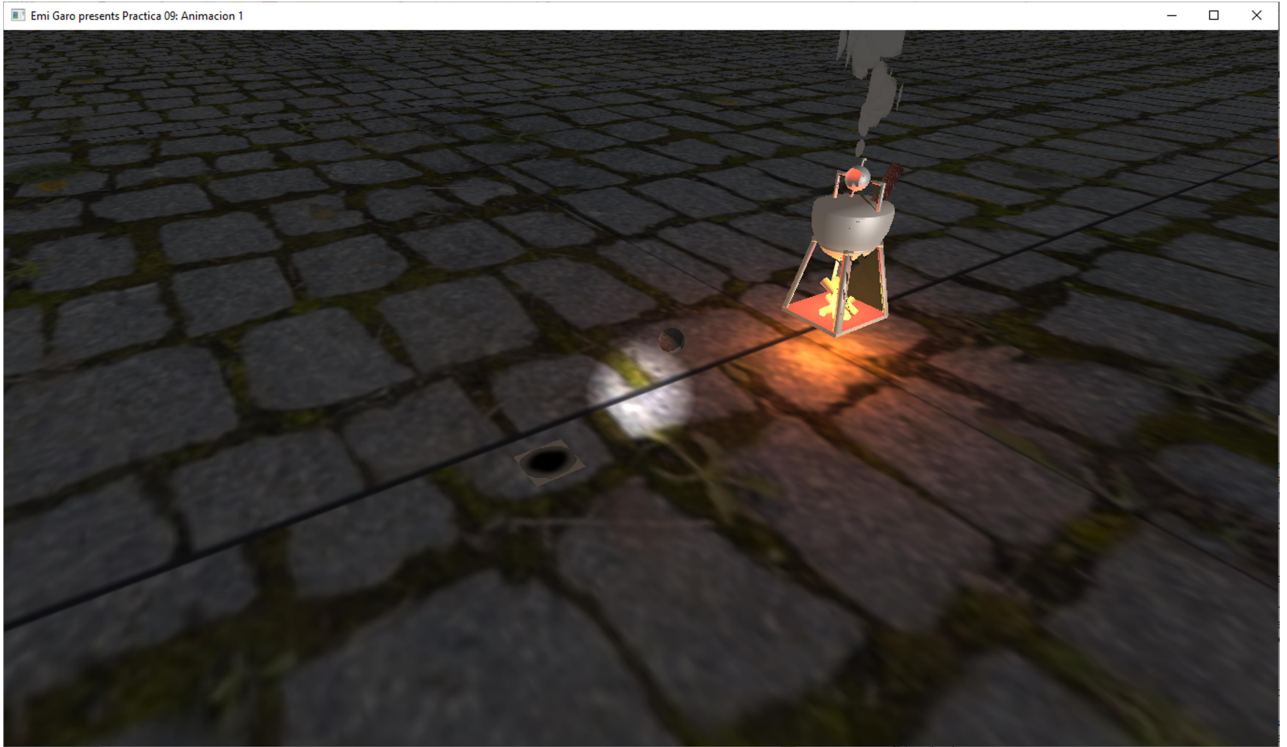
    // En el segundo rebote, corregir rumbo directo al hueco
    // para que el tercer bote caiga exactamente en el
    if (rebotes == 2) {
        float dx = POS_HUECO.x - esfera_x;
        float dz = POS_HUECO.z - esfera_z;
        float t_corto = 0.6f;
        esfera_vx = dx / t_corto;
        esfera_vz = dz / t_corto;
        esfera_vy = (0.5f * GRAVEDAD * t_corto * t_corto) / t_corto;
    }
    // arco bajo
}
// Entra al hueco solo despues del segundo rebote (tercer bote)
if (rebotes >= 2) {
    float dx = esfera_x - POS_HUECO.x;
    float dz = esfera_z - POS_HUECO.z;
    float distH = sqrt(dx * dx + dz * dz);
    if (distH < 0.8f && esfera_y <= piso + 0.15f) {
        estado_esfera = 2;
    }
}
}

```

Coordinación de los tres sistemas

La función actualizarCiclo(dt) avanza simultáneamente los cuatro sistemas (fuego, humo, brazo, esfera) cada frame. Las transiciones de estado están enlazadas por eventos:

- El fuego se enciende al inicio y permanece activo durante todo el ciclo.
- El humo al máximo (estado 2 → intensidad 1.0) dispara el brazo (estado 0 → 1).
- El brazo tensado (estado 1) pasa a lanzamiento (estado 2) cuando alcanza -65°.
- El brazo en lanzamiento (estado 2 → >30°) suelta la esfera y la envía en tiro parabólico.
- La esfera después de 2 rebotes entra al hueco (estado 2), cae, reaparece en el brazo (estado 3 → 0) y dispara el apagado del humo (estado 3).
- El ciclo se reinicia automáticamente cada vez que la esfera reaparece.



Conclusión

Esta práctica fue un buen cierre de los conceptos vistos durante el semestre. A diferencia de prácticas anteriores donde la animación se basaba en banderas y condicionales sencillos, aquí tuvimos que usar máquinas de estados, integración numérica para la física y funciones senoidales para el sistema de partículas del humo. Lo más interesante fue ver cómo cuatro sistemas independientes (fuego, humo, brazo, esfera) pueden coordinarse mediante eventos para producir un comportamiento cíclico complejo.

El proceso de debugueado también fue valioso, ya que los errores fueron errores difíciles de detectar visualmente pero sencillos de corregir una vez identificados. Esto refuerza la importancia de verificar el layout de vértices y el pipeline completo antes de concluir que algo es un bug de lógica.

La física del rebote con dos botes naturales y un tercer bote dirigido al hueco resuelve elegantemente el problema de garantizar que la esfera entre al hueco sin que la animación parezca forzada. Con esto cerramos las herramientas básicas para construir escenas dinámicas: modelado, texturas, iluminación, jerarquía y animación compleja con física simple.

Referencias

- Apuntes de la clase de teoría y laboratorio de Computación Gráfica e Interacción Humano Computadora.
- De Vries, J. (n.d.). Blending. LearnOpenGL. <https://learnopengl.com/Advanced-OpenGL/Blending>
- De Vries, J. (n.d.). Particles. LearnOpenGL. <https://learnopengl.com/In-Practice/2D-Game/Particles>
- Meiri, E. Tutorial 28 – Particle System. OGLdev. <https://ogldev.org/www/tutorial28/tutorial28.html>
- Martell, R. (2016). Sphere.cpp / Sphere.h. Material de clase, UNAM Facultad de Ingeniería.